MongoDB Indexes Lab

Indexes, query planning, explain output, geospatial and full-text search.
Prerequisites: MongoDB Lab Worksheet completed

Course	Tools	Suggested files
ACIT 1630 / Database Systems	Docker Desktop, terminal, mongosh	Word/PDF with answers and screenshots

Learning outcomes

- Understand why indexes exist and how they affect query performance.
- Create single-field and compound indexes and choose the right type for a query.
- Use explain() to read a query plan and identify collection scans versus index scans.
- Know when an index helps and when it adds unnecessary overhead.
- Index fields inside embedded objects and arrays.
- Understand index cardinality and why it matters for selectivity.
- Use geospatial indexes to query location data.
- Use text indexes to search string content.

Before you begin: complete the MongoDB Lab Worksheet first. This lab reuses the school database and students collection. Start the container with: `docker run -d --name mongodb-lab -p 27017:27017 -v mongodb_data:/data/db mongo:8.0`, then connect with: `docker exec -it mongodb-lab mongosh`, and run `use school`.

Part 1 — Why indexes exist

Without an index, MongoDB must scan every document in a collection to answer a query. This is called a collection scan (COLLSCAN). With an index, MongoDB follows a B-tree structure to jump directly to the matching documents. The difference is invisible on a five-document collection but significant at scale: a collection with one million documents may take seconds to COLLSCAN but microseconds to use an index.

The students collection from the earlier lab has five documents. To see the problem clearly, load more data first.

Load sample data for this lab

```
use school

db.students.insertMany([
  { name: "Diana",   age: 23, course: "SE", city: "Richmond",  gpa: 3.8 },
  { name: "Eve",     age: 20, course: "DB", city: "Delta",     gpa: 3.2 },
  { name: "Frank",   age: 24, course: "AI", city: "Surrey",    gpa: 3.5 },
  { name: "Grace",   age: 22, course: "SE", city: "Burnaby",   gpa: 3.9 },
  { name: "Hank",    age: 21, course: "DB", city: "Vancouver", gpa: 2.8 },
  { name: "Ivy",     age: 25, course: "AI", city: "Surrey",    gpa: 3.1 },
  { name: "Jack",    age: 20, course: "SE", city: "Delta",     gpa: 3.6 },
])
```

Purpose: Inserts additional student documents so the collection has enough variety to demonstrate index selectivity and cardinality differences.

Inputs/Outputs: An array of seven documents, each with name, age, course, city, and a new gpa field. Output: acknowledged: true with seven insertedIds.

Variations

```
// Check how many documents are in the collection:
db.students.countDocuments()
```

Part 2 — Reading query plans with explain()

explain() tells you how MongoDB executed a query: whether it scanned the whole collection, which index it used, how many documents it examined versus how many it returned. Always run explain() before and after creating an index to confirm the improvement.

See a collection scan (no index yet)

```
db.students.find({ course: "DB" }).explain("executionStats")
```

Purpose: Executes the query and returns a detailed plan showing how MongoDB satisfied it. The string "executionStats" requests runtime numbers on top of the plan.

Inputs/Outputs: Input: the same filter you would pass to find(). Output: a large JSON object. The critical fields to read are: winningPlan.stage — COLLSCAN means every document was examined; totalDocsExamined — how many documents MongoDB looked at; totalKeysExamined — how many index entries were checked (0 if no index was used); nReturned — how many documents matched; executionTimeMillis — wall-clock time.

Variations

```
// Minimal plan only (no runtime stats, faster):
db.students.find({ course: "DB" }).explain()

// All three verbosity levels:
db.students.find({ course: "DB" }).explain("queryPlanner")   // plan only
db.students.find({ course: "DB" }).explain("executionStats") // plan + stats
db.students.find({ course: "DB" }).explain("allPlansExecution") // all candidates
```

queryPlanner is the default and shows the winning plan without running it. executionStats actually executes the query and returns counts. allPlansExecution runs all candidate plans simultaneously and is useful for diagnosing why MongoDB chose one plan over another.

SQLite equivalent

```
-- SQLite EXPLAIN QUERY PLAN shows the access strategy:
EXPLAIN QUERY PLAN SELECT * FROM students WHERE course = 'DB';

-- Output: SCAN students   (means full table scan, no index used)
-- After creating an index you would see: SEARCH students USING INDEX ...
```

SQLite's EXPLAIN QUERY PLAN is the direct equivalent. Both tools show whether the engine will scan or seek, and both are the correct first step before and after adding an index.

Key fields in explain() output to focus on

```
winningPlan.stage: COLLSCAN = no index used | IXSCAN = index used | FETCH = fetching docs after index
totalDocsExamined: should be close to nReturned after indexing
totalKeysExamined:  index entries read (ideally equals nReturned)
nReturned:          documents that matched the query
executionTimeMillis: total time in milliseconds
```

Task

- □ Run the explain on db.students.find({ course: "DB" }) and note the stage and totalDocsExamined values.
- □ Record those numbers — you will compare them after creating an index in Part 3.

Part 3 — Single-field indexes

A single-field index covers one field. MongoDB sorts the field values into a B-tree so queries that filter, sort, or range-scan on that field can skip most documents.

Create a single-field index on course

```
db.students.createIndex({ course: 1 })
```

Purpose: Builds an ascending B-tree index on the course field. Queries that filter on course, sort on course, or use course in a range will now use this index instead of scanning the collection.

Inputs/Outputs: The key document { course: 1 } specifies the field and direction: 1 = ascending, -1 = descending. For a single-field index the direction only affects sort order, not filtering performance. Output: an object containing the name of the created index, e.g. course_1.

Variations

```
// Descending index (useful when queries always sort descending):
db.students.createIndex({ age: -1 })

// Named index (easier to drop later):
db.students.createIndex({ city: 1 }, { name: "idx_city" })

// Unique index (enforces uniqueness like a SQL UNIQUE constraint):
db.students.createIndex({ name: 1 }, { unique: true })

// Sparse index (only indexes documents where the field exists):
db.students.createIndex({ gpa: 1 }, { sparse: true })

// List all indexes on the collection:
db.students.getIndexes()

// Drop an index by name:
db.students.dropIndex("course_1")
```

A unique index rejects any insert or update that would create a duplicate value, exactly like a SQL UNIQUE constraint. A sparse index skips documents that do not have the indexed field, which saves space when a field is optional.

SQLite equivalent

```
-- Create an ascending index:
CREATE INDEX idx_course ON students(course);
```

```
-- Unique index:
CREATE UNIQUE INDEX idx_name ON students(name);

-- List all indexes (SQLite):
PRAGMA index_list('students');

-- Drop an index:
DROP INDEX idx_course;
```

SQL CREATE INDEX and MongoDB createIndex() serve the same purpose and use very similar syntax. One difference: in MongoDB the direction (1 or -1) is part of the index definition; in SQLite you write ASC or DESC after the column name (ASC is the default).

Task

- □ Create the index: db.students.createIndex({ course: 1 })
- □ Re-run explain("executionStats") on the same find({ course: "DB" }) query.
- □ Compare totalDocsExamined and stage to your notes from Part 2.
- □ Run db.students.getIndexes() and identify the two indexes listed (the _id index is always present).

Part 4 — Compound indexes

A compound index covers multiple fields in a single B-tree. It can satisfy queries that filter or sort on any left-prefix of those fields. The order of fields in a compound index matters: an index on { course, age } can support queries on course alone or on course + age together, but not on age alone.

The rule of thumb for field order in a compound index is ESR: Equality fields first, then Sort fields, then Range fields.

Create a compound index on course and age

```
db.students.createIndex({ course: 1, age: -1 })
```

Purpose: Builds a compound index that sorts documents first by course ascending, then by age descending within each course group. This index efficiently supports queries that filter on course and sort by age without an additional sort step.

Inputs/Outputs: The key document lists fields in priority order. Direction per field: 1 ascending, -1 descending. Output: the index name course_1_age_-1.

Variations

```
// Query that uses the full compound index (filter + sort covered):
db.students.find({ course: "DB" }).sort({ age: -1 }).explain("executionStats")

// Query that uses only the prefix (course field):
db.students.find({ course: "SE" }).explain("executionStats")

// Query that CANNOT use the index (age alone, not a prefix):
db.students.find({ age: { $gt: 21 } }).explain("executionStats")

// ESR example: equality on course, sort on name, range on age:
db.students.createIndex({ course: 1, name: 1, age: 1 })
```

The third example will show COLLSCAN because age is not a left-prefix of the { course, age } index. To support range queries on age alone you would need a separate { age: 1 } index. MongoDB can use at most one index per query stage (unless it performs an index intersection, which is rare).

SQLite equivalent

```
-- Compound index in SQLite:
CREATE INDEX idx_course_age ON students(course ASC, age DESC);

-- Verify the index is used:
EXPLAIN QUERY PLAN SELECT * FROM students WHERE course = 'DB' ORDER BY age DESC;

-- A query on age alone will NOT use idx_course_age:
EXPLAIN QUERY PLAN SELECT * FROM students WHERE age > 21;
```

SQLite compound indexes follow the same left-prefix rule. A query must include the leftmost column(s) of the index in its WHERE clause for the index to be used.

Task

- □ Create the compound index: db.students.createIndex({ course: 1, age: -1 })
- □ Run explain("executionStats") on db.students.find({ course: "DB" }).sort({ age: -1 }) and confirm the stage is IXSCAN.
- □ Run explain on db.students.find({ age: { $gt: 21 } }) and confirm it is still COLLSCAN.
- □ Explain in one sentence why the compound index cannot support an age-only query.

Part 5 — How $ operators interact with indexes

Not all query operators use indexes equally. Knowing which operators are index-friendly helps you write faster queries and design better indexes.

Test $gt, $in, and $regex with explain()

```
// $gt — range, uses index efficiently:
db.students.find({ age: { $gt: 21 } }).explain("executionStats")

// $in — equality on a list, uses index (one seek per value):
db.students.find({ course: { $in: ["DB", "AI"] } }).explain("executionStats")

// $regex anchored — can use index for prefix scan:
db.students.find({ name: { $regex: /^G/ } }).explain("executionStats")

// $regex unanchored — cannot use index, always COLLSCAN:
db.students.find({ name: { $regex: /ace/ } }).explain("executionStats")
```

Purpose: Demonstrates which operators can leverage an index and which cannot. An index on the filtered field must exist for these tests to show IXSCAN.

Inputs/Outputs: Ensure the { course: 1 } index from Part 3 and an { age: 1 } index exist before running. Create the age index with db.students.createIndex({ age: 1 }) if needed. Output: explain output for each query showing the winning stage.

Variations

```
// $exists with sparse index — only useful when field is often absent:
db.students.find({ gpa: { $exists: true } }).explain("executionStats")

// $ne — negation, MongoDB usually avoids indexes for negations:
db.students.find({ course: { $ne: "AI" } }).explain("executionStats")
```

```
// $and — both fields indexed separately, only one index used per stage:
db.students.find({ $and: [{ course: "DB" }, { age: { $gt: 20 } }] }).explain("executionStats")

// Best option: compound index covers both fields:
db.students.find({ course: "DB", age: { $gt: 20 } }).explain("executionStats")
```

$ne and $nin are generally not index-friendly because they match the majority of documents. $and with two separate single-field indexes may trigger index intersection but a compound index is usually more efficient. Anchored regex (/^pattern/) can use an index prefix scan; unanchored regex must COLLSCAN.

SQLite equivalent

```
-- Range ($gt equivalent):
EXPLAIN QUERY PLAN SELECT * FROM students WHERE age > 21;

-- IN list ($in equivalent):
EXPLAIN QUERY PLAN SELECT * FROM students WHERE course IN ('DB', 'AI');

-- Prefix LIKE (anchored $regex equivalent):
EXPLAIN QUERY PLAN SELECT * FROM students WHERE name LIKE 'G%';

-- Non-prefix LIKE (unanchored $regex) — always full scan:
EXPLAIN QUERY PLAN SELECT * FROM students WHERE name LIKE '%ace%';
```

SQLite follows identical rules: range operators and IN lists use indexes; LIKE with a leading wildcard (%) cannot use a B-tree index and always scans the table.

Task

- □ Create db.students.createIndex({ age: 1 }) and db.students.createIndex({ name: 1 }).
- □ Run explain on db.students.find({ name: { $regex: /^G/ } }) — confirm IXSCAN.
- □ Run explain on db.students.find({ name: { $regex: /ace/ } }) — confirm COLLSCAN.
- □ Write one sentence explaining why an unanchored regex cannot use a B-tree index.

Part 6 — Indexing embedded objects and arrays

MongoDB documents can contain nested objects (sub-documents) and arrays. Both can be indexed using dot notation. An index on an array field is called a multikey index and MongoDB creates one automatically when it detects an array value.

Insert documents with nested objects and arrays

```
db.students.updateMany({}, [
  { $set: { address: { city: "$city", province: "BC" } } }
])

db.students.updateOne({ name: "Alice" }, { $set: { tags: ["honours", "scholarship"] } })
db.students.updateOne({ name: "Bob" },   { $set: { tags: ["exchange"] } })
db.students.updateOne({ name: "Diana" }, { $set: { tags: ["honours", "exchange"] } })
```

Purpose: Adds an address sub-document to every student using an aggregation pipeline update (the $city refers to the existing city field), and adds a tags array to three students. This sets up the data needed to demonstrate dot-notation indexing and multikey indexes.

Inputs/Outputs: updateMany with an aggregation pipeline (square brackets as the second argument) allows field references inside $set. Output: matchedCount and modifiedCount for each update.

Variations

```
// Query nested field without an index (will COLLSCAN):
db.students.find({ "address.province": "BC" }).explain("executionStats")
```

Index a nested object field with dot notation

```
db.students.createIndex({ "address.city": 1 })
```

Purpose: Creates an index on the city field inside the address sub-document. Queries that filter on address.city will now use an IXSCAN. The field path uses dot notation just as it does in queries.

Inputs/Outputs: The field name must be quoted because it contains a dot. Output: the index name "address.city_1". Queries must also use the same dot path: { "address.city": "Surrey" }.

Variations

```
// Confirm the index is used:
db.students.find({ "address.city": "Surrey" }).explain("executionStats")

// Index two levels deep:
db.students.createIndex({ "contact.email.domain": 1 })
```

SQLite equivalent

```
-- SQLite has no native nested-object support.
-- The closest equivalent is a generated column (SQLite 3.31+):
ALTER TABLE students ADD COLUMN province TEXT
  GENERATED ALWAYS AS (json_extract(address_json, '$.province')) STORED;
CREATE INDEX idx_province ON students(province);
```

SQLite stores JSON as TEXT and uses json_extract() to reach nested values. An index on a generated column that extracts the nested field achieves the same result as MongoDB's dot-notation index.

Task

- □ Run the two updateMany/updateOne commands to add address and tags fields.
- □ Create the index: db.students.createIndex({ "address.city": 1 })
- □ Confirm with explain that db.students.find({ "address.city": "Surrey" }) uses IXSCAN.

Multikey index on an array field

```
db.students.createIndex({ tags: 1 })
```

Purpose: Creates a multikey index on the tags array. MongoDB automatically creates one index entry per element in the array, so a query for any single tag value hits the index. The explain output will show isMultiKey: true.

Inputs/Outputs: The syntax is identical to a regular single-field index. MongoDB detects the array and switches to multikey mode automatically. Output: index name tags_1. A multikey index cannot be combined with another multikey field in the same compound index.

Variations

```
// Query any element in the array — uses the multikey index:
db.students.find({ tags: "honours" }).explain("executionStats")
```

```
// Query where ALL elements match ($all) — also uses the index:
db.students.find({ tags: { $all: ["honours", "exchange"] } }).explain("executionStats")

// Query exact array (order-sensitive) — may not use index efficiently:
db.students.find({ tags: ["honours", "scholarship"] })
```

$elemMatch is preferred when matching a condition against a single element of an array of sub-documents, and it also benefits from a multikey index. Querying for an exact array match (the whole array in order) bypasses the multikey index and does a full scan.

SQLite equivalent

```
-- SQLite has no native array column.
-- Common patterns: a junction table, or JSON array with json_each():
CREATE TABLE student_tags (student_id INTEGER, tag TEXT);
CREATE INDEX idx_tag ON student_tags(tag);

-- Query with junction table (equivalent to multikey index query):
SELECT s.* FROM students s JOIN student_tags t ON s.id = t.student_id WHERE t.tag = 'honours';
```

The relational equivalent of a multikey index is a separate junction table with an index on the tag column. MongoDB's multikey index handles this without a second table.

Task

- □ Create the index: db.students.createIndex({ tags: 1 })
- □ Run explain on db.students.find({ tags: "honours" }) and confirm isMultiKey: true.
- □ Run explain on db.students.find({ tags: { $all: ["honours", "exchange"] } }) and note the stage.

Part 7 — Index cardinality

Cardinality is the number of distinct values a field has across the collection. A field with high cardinality (like name or email) has many unique values; a field with low cardinality (like a boolean or a status with three possible values) has few. Indexes on high-cardinality fields are more selective — they narrow down the result set significantly — and are therefore more useful.

Compare cardinality across fields

```
// Count distinct values per field:
db.students.distinct('course').length
db.students.distinct('city').length
db.students.distinct('name').length
db.students.distinct('age').length
```

Purpose: Shows the number of distinct values for each field. This is a quick way to estimate cardinality before deciding whether an index on that field is worthwhile.

Inputs/Outputs: distinct() takes a field name and returns an array of unique values. .length gives the count. Output: a number for each field.

Variations

```
// See the actual distinct values (useful for low-cardinality fields):
db.students.distinct('course')
```

```
// Estimate index selectivity: ratio of distinct values to total documents.
// High ratio (close to 1.0) = high cardinality = good index candidate.
// Low ratio (close to 0.0) = low cardinality = index may not help.
const total = db.students.countDocuments()
const distinct = db.students.distinct('course').length
print('Selectivity:', (distinct / total).toFixed(2))
```

A boolean field (true/false) has cardinality 2. If half the documents are true, an index returns half the collection — nearly as expensive as a COLLSCAN. MongoDB may ignore such indexes and COLLSCAN instead. The query planner makes this decision automatically based on cost estimates.

SQLite equivalent

```
-- Count distinct values per column:
SELECT COUNT(DISTINCT course) FROM students;
SELECT COUNT(DISTINCT city)   FROM students;
SELECT COUNT(DISTINCT name)   FROM students;

-- Selectivity ratio:
SELECT CAST(COUNT(DISTINCT course) AS REAL) / COUNT(*) AS selectivity FROM students;
```

Task

- □ Run db.students.distinct('course').length and db.students.distinct('name').length.
- □ Which field has higher cardinality?
- □ Which field would make a more useful index, and why?
- □ Calculate the selectivity ratio for course. Is it above or below 0.5?

Part 8 — When to index and when not to

Index — YES	Index — NO / use with caution
Fields frequently used in query filters (find, updateOne, deleteOne)	Fields rarely or never queried
Fields used in sort operations on large result sets	Collections with fewer than a few thousand documents (COLLSCAN is fast enough)
High-cardinality fields (email, name, ID, timestamp)	Low-cardinality fields (boolean, status with 2–3 values)
Fields used in range queries ($gt, $lt, $between)	Fields only queried with unanchored regex (the index is never used)
Foreign-key-style reference fields joining collections	Write-heavy collections where index maintenance slows inserts
Fields covered by covering index queries (all needed fields in index)	Fields that are mostly null or missing (use sparse index instead)

Every index has a cost: it consumes disk space and RAM, and MongoDB must update every applicable index on every insert, update, and delete. A collection with 20 unnecessary indexes may actually perform worse on writes than one with 3 well-chosen indexes.

Check index usage statistics

```
db.students.aggregate([{ $indexStats: {} }])
```

Purpose: Returns usage statistics for each index on the collection: how many times each index has been used since the mongod process started. Use this to identify indexes that are never accessed and can be dropped.

Inputs/Outputs: No pipeline arguments needed. Output: one document per index containing name, key, and accesses.ops (the number of times this index was used). An accesses.ops of 0 for a non-_id index is a strong sign the index is unused and should be reviewed for removal.

Variations

```
// List all indexes with their sizes:
db.students.stats().indexSizes

// Total index size for the collection:
db.students.totalIndexSize()

// Drop an index that is never used:
db.students.dropIndex("course_1")

// Drop all non-_id indexes (use with caution in production):
db.students.dropIndexes()
```

Task

- □ Run db.students.aggregate([{ $indexStats: {} }]) and list all indexes and their access counts.
- □ Run db.students.stats().indexSizes and note which index takes the most space.
- □ Which index has the lowest access count? Would you drop it in production? Why or why not?

Part 9 — Types of indexes

MongoDB supports several index types beyond the default B-tree. The right type depends on the data shape and the queries you need to support.

Type	Use case	createIndex() syntax
Single-field	Most queries on one field	{ field: 1 } or { field: -1 }
Compound	Queries on multiple fields or covered queries	{ field1: 1, field2: -1 }
Multikey	Queries on array field elements (auto-created)	{ arrayField: 1 }
Text	Full-text keyword search on string fields	{ field: "text" }
2dsphere	Geospatial queries on GeoJSON (lat/lng points, polygons)	{ location: "2dsphere" }
2d	Legacy flat-plane coordinate queries (rarely used)	{ coords: "2d" }
Hashed	Equality queries only; used for hash-based sharding	{ field: "hashed" }
Wildcard	Index all fields or a subtree (schema-flexible collections)	{ "$**": 1 }
Sparse	Only index documents where the field exists	{ field: 1 }, { sparse: true }

Partial	Only index documents matching a filter expression	{ field: 1 }, { partialFilterExpression: { status: "active" } }
TTL	Auto-delete documents after a time period (e.g. sessions)	{ createdAt: 1 }, { expireAfterSeconds: 3600 }

Part 10 — Geospatial indexes

A 2dsphere index stores coordinates in GeoJSON format and supports queries like: find all documents within X metres of a point, find all documents inside a polygon, or find the nearest N documents to a location. This is the correct index type for real-world latitude/longitude data.

Insert documents with GeoJSON location data

```
db.campuses.insertMany([
  { name: "BCIT Burnaby",    location: { type: "Point", coordinates: [-122.9996, 49.2488] } },
  { name: "BCIT Downtown",   location: { type: "Point", coordinates: [-123.1139, 49.2827] } },
  { name: "SFU Burnaby",     location: { type: "Point", coordinates: [-122.9199, 49.2788] } },
  { name: "UBC Vancouver",   location: { type: "Point", coordinates: [-123.2460, 49.2606] } },
  { name: "Langara College", location: { type: "Point", coordinates: [-123.1090, 49.2237] } },
])
```

Purpose: Creates a new campuses collection with GeoJSON Point documents. Each document stores coordinates as [longitude, latitude] — GeoJSON always uses longitude first, which is the opposite of the common lat/lng convention. Make sure to follow this order or queries will return incorrect results.

Inputs/Outputs: Each location field is a GeoJSON object with a type property (always the string Point for a single coordinate) and a coordinates array of [longitude, latitude]. Output: acknowledged: true with five insertedIds.

Variations

```
// GeoJSON also supports Polygon for area shapes:
db.zones.insertOne({ name: "Surrey Zone", boundary: {
  type: "Polygon",
  coordinates: [[[-122.85, 49.10], [-122.75, 49.10], [-122.75, 49.20], [-122.85, 49.20], [-122.85, 49.10]]]
}})
```

Create a 2dsphere index

```
db.campuses.createIndex({ location: "2dsphere" })
```

Purpose: Builds a 2dsphere index on the location field. This index is required before any geospatial query operator ($near, $geoWithin, $geoIntersects) will work. The value "2dsphere" (a string, not a number) tells MongoDB to use spherical geometry appropriate for Earth-surface coordinates.

Inputs/Outputs: Field name and the string "2dsphere" as the value. Output: the index name location_2dsphere.

SQLite equivalent

```
-- SQLite has no built-in geospatial support.
-- The SpatiaLite extension adds geospatial functions:
SELECT load_extension('mod_spatialite');
SELECT CreateSpatialIndex('campuses', 'location');
```

```
-- Without SpatiaLite, a bounding-box approximation is possible:
CREATE INDEX idx_lat_lng ON campuses(latitude, longitude);
SELECT * FROM campuses WHERE latitude BETWEEN 49.20 AND 49.30
  AND longitude BETWEEN -123.00 AND -122.90;
```

Standard SQLite has no spherical geometry. The bounding-box SQL approximation above finds candidates but does not account for Earth's curvature — it is inaccurate for large distances. SpatiaLite (an extension) provides proper geospatial support similar to PostGIS.

Find campuses near a location ($near)

```
db.campuses.find({ location: { $near: {
  $geometry: { type: "Point", coordinates: [-122.9996, 49.2488] },
  $maxDistance: 5000
}}})
```

Purpose: Returns documents sorted by distance from the given point, closest first. $maxDistance limits results to documents within 5000 metres. The 2dsphere index is required for $near.

Inputs/Outputs: $geometry is the reference point in GeoJSON format. $maxDistance and $minDistance are optional and in metres. Output: matching documents ordered by ascending distance from the query point.

Variations

```
// $nearSphere is identical to $near for 2dsphere indexes:
db.campuses.find({ location: { $nearSphere: { $geometry: { type: "Point", coordinates: [-122.9996, 49.2488] },
$maxDistance: 5000 }}})

// Find all campuses within a circular area ($geoWithin + $centerSphere):
// Radius is in radians: metres / 6378137
db.campuses.find({ location: { $geoWithin: { $centerSphere: [[-122.9996, 49.2488], 5000/6378137] }}})

// Count campuses within 10 km:
db.campuses.countDocuments({ location: { $near: { $geometry: { type: "Point", coordinates: [-122.9996,
49.2488] }, $maxDistance: 10000 }}})
```

$near returns results in distance order and requires the 2dsphere index. $geoWithin returns all documents inside a shape (in any order) and also benefits from the index. Use $geoWithin when you need all results inside a boundary without caring about sort order.

Task

- □ Insert the five campus documents and create the 2dsphere index.
- □ Run the $near query and list the campuses returned within 5 km of BCIT Burnaby.
- □ Change $maxDistance to 20000 and note which additional campuses appear.
- □ Write one sentence explaining why GeoJSON uses [longitude, latitude] instead of [latitude, longitude].

Part 11 — Full-text search with text indexes

A text index tokenises string fields into individual words (stems them to their root form) and builds an inverted index mapping each word to the documents that contain it. This enables keyword search across one or more string fields with the $text operator. Only one text index is allowed per collection, but it can cover multiple fields.

Insert documents for text search

```
db.courses.insertMany([
  { code: "ACIT1630", title: "Database Systems",        description: "Relational and NoSQL databases, SQL, MongoDB, data modelling." },
  { code: "ACIT2420", title: "Linux System Admin",      description: "Linux shell scripting, user management, networking, and services." },
  { code: "ACIT3620", title: "Advanced Databases",      description: "Query optimisation, indexing strategies, replication, and sharding in MongoDB." },
  { code: "ACIT2521", title: "Data Structures",         description: "Arrays, linked lists, trees, graphs, sorting and searching algorithms." },
  { code: "ACIT4850", title: "Machine Learning",        description: "Supervised learning, neural networks, model evaluation and deployment." },
])
```

Purpose: Creates a courses collection with descriptive text fields that demonstrate keyword search. The title and description fields will be covered by the text index.

Inputs/Outputs: Five course documents each with code, title, and description. Output: acknowledged: true with five insertedIds.

Create a text index covering multiple fields

```
db.courses.createIndex({ title: "text", description: "text" })
```

Purpose: Creates a single text index that covers both the title and description fields. MongoDB tokenises every word in both fields and builds an inverted index. The string "text" as the value (instead of 1 or -1) signals MongoDB to build a text index rather than a B-tree.

Inputs/Outputs: One or more fields with the value "text". Output: the index name title_text_description_text. Only one text index can exist per collection.

Variations

```
// Cover all string fields with a wildcard text index:
db.courses.createIndex({ "$**": "text" })

// Weight fields (title matches count more than description):
db.courses.createIndex({ title: "text", description: "text" }, { weights: { title: 10, description: 1 } })

// Set a default language (affects stemming — 'running' → 'run'):
db.courses.createIndex({ description: "text" }, { default_language: "english" })
```

Weights let you boost the relevance score of matches in certain fields. A match in a field with weight 10 contributes ten times more to the score than a match in a field with weight 1. The textScore meta-field in a projection exposes the relevance score.

Search with $text and $search

```
// Find courses mentioning "MongoDB":
db.courses.find({ $text: { $search: "MongoDB" } })

// Find courses about databases OR indexing (space = OR):
db.courses.find({ $text: { $search: "database indexing" } })
```

```
// Exact phrase search (quoted with escaped quotes):
db.courses.find({ $text: { $search: "\"data modelling\"" } })

// Exclude a word (prefix with -):
db.courses.find({ $text: { $search: "database -relational" } })

// Sort by relevance score:
db.courses.find({ $text: { $search: "database" } }, { score: { $meta: "textScore" } }).sort({ score: { $meta:
"textScore" } })
```

Purpose: $text performs keyword search against the text index. The $search string can contain individual words (OR semantics), exact phrases (in double quotes), and excluded words (prefixed with -).

Inputs/Outputs: $text takes a $search string and optional $language and $caseSensitive flags. $meta: "textScore" retrieves the relevance score computed by the text index. Output: matching documents. Order is not guaranteed unless you sort by textScore.

Variations

```
// Case-sensitive search (default is case-insensitive):
db.courses.find({ $text: { $search: "MongoDB", $caseSensitive: true } })

// Combine text search with another filter:
db.courses.find({ $text: { $search: "database" }, code: { $regex: /^ACIT1/ } })

// Count matching documents:
db.courses.countDocuments({ $text: { $search: "Linux" } })
```

Text search is case-insensitive and diacritic-insensitive by default. MongoDB stems words (e.g. 'indexing' matches 'index') based on the configured language. You cannot combine two $text operators in a single query — if you need to search across collections, use separate queries.

SQLite equivalent

```
-- SQLite FTS5 (Full-Text Search extension, built into SQLite):
CREATE VIRTUAL TABLE courses_fts USING fts5(title, description, content='courses', content_rowid='id');

-- Populate the FTS index:
INSERT INTO courses_fts(courses_fts) VALUES('rebuild');

-- Search for 'MongoDB':
SELECT * FROM courses_fts WHERE courses_fts MATCH 'MongoDB';

-- Phrase search:
SELECT * FROM courses_fts WHERE courses_fts MATCH '"data modelling"';

-- Sort by relevance (BM25 score):
SELECT *, bm25(courses_fts) AS score FROM courses_fts WHERE courses_fts MATCH 'database' ORDER BY
score;
```

SQLite's FTS5 module is the direct equivalent of MongoDB's text index. Both tokenise text, support phrase search with quotes and exclusions with -, and expose a relevance score (textScore in MongoDB, bm25() in SQLite FTS5).

Task

- □ Create the text index: db.courses.createIndex({ title: "text", description: "text" })
- □ Search for "MongoDB" and note which courses are returned.
- □ Search for "database -relational" and explain what the minus sign did.
- □ Sort results by relevance score and write down which course ranked highest for "database indexing".
- □ Run explain() on a $text query and note the stage name (IXSCAN with a text index shows as TEXT).